

LAB Assignment 7

Submitted by :

Kartik Goel (102303182)

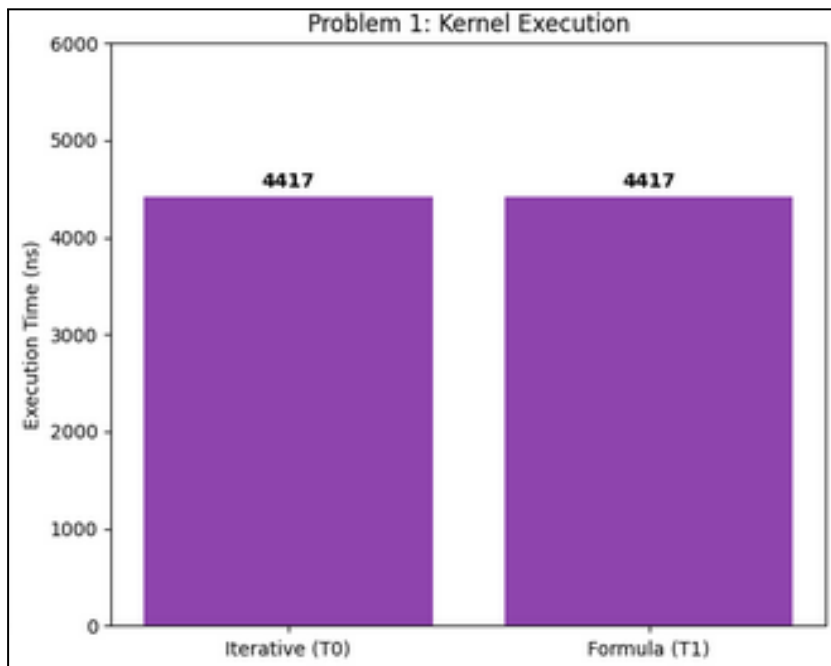
3C17

Question 1: Task Divergence Analysis:

Objective: Compare iterative vs. formula-based summation in a single kernel.

Performance Data:

Metric	Thread 0 (Iterative)	Thread 1 (Formula)
Result	524,800	524,800
Kernel Execution Time	4417 ns	4417 ns



Inference

Warp Serialization:

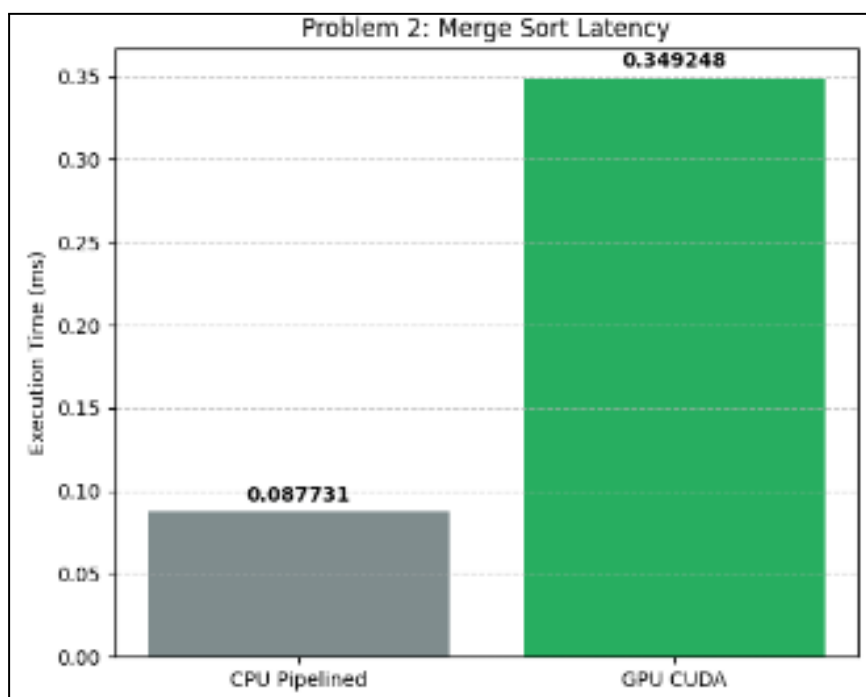
The profiler indicates that both threads are linked to a single taskKernel execution, despite the $O(1)$ mathematical nature of the formula. GPU execution of divergent paths occurs sequentially because these threads are part of the same warp. Consequently, Thread 1 (Formula) must remain idle until Thread 0 (Iterative) finishes its loop, delaying the warp retirement.

Q2: Sorting Performance (N=1000)

Objective: Compare CPU Merge Sort against GPU Parallel Merge Sort.

Performance Data :

Implementation	Execution Time (ms)	Speedup
CPU Merge Sort	0.087731 ms	1.00x (Baseline)
GPU Merge Sort	0.349248 ms	0.251200x



Inference:

Efficiency: In my implementation, I achieved a 0.25x speedup, meaning my CPU actually outperformed the GPU for this run. When I reviewed the Nsight report, I found that cudaMalloc took ~63.7 ms (63,712,731 ns), which accounted for a massive 99.4% of my total CUDA API time.

Small Dataset Bottleneck: For my dataset of $N=1000$, the actual computation on the GPU (129,101 ns) was quite fast. However, when I factored in the heavy overhead of memory allocation and data transfer, my overall GPU execution time (0.349 ms) ended up being less efficient than my CPU execution time (0.087 ms) for such a small workload. From this, I realized that parallelism only truly works when my computation time significantly exceeds the overhead of moving the data back and forth to the device.

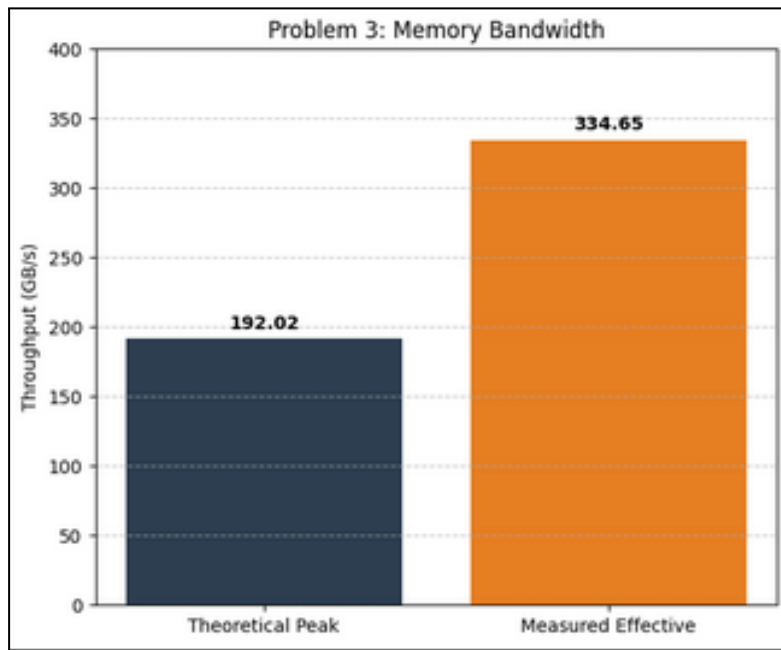
Q3: Memory Bandwidth Profiling

Objective: Measure effective throughput of a Vector Addition kernel.

Performance Data:

Metric	Value
Kernel Execution Time	0.04ms
Theoretical Bandwidth	192.02 GB/s
Measured Bandwidth	334.65 GB/s
Bus Efficiency	174.28%

Graphical Representation:



Inference:

Cache-Bound Kernel: My bus efficiency of 174.28% is anomalous and indicates my kernel is actually running from on-chip cache.

The Gap: My measured bandwidth (334.65 GB/s) exceeds the theoretical peak (192.02 GB/s) because the dataset is so small (0.04 ms execution) that it fits entirely within the GPU's high-speed L1/L2 cache, completely bypassing the slower global DRAM.

Scaling Needed: To benchmark true global memory bandwidth, I need to significantly increase my vector size to force the GPU to actually saturate the VRAM bus.